

Day7.

파이썬 Streamlit GUI

웹프로그래밍



목 차

1. 파이썬 GUI		3
2. Streamlit 이란?		4
3. 관련 라이브러리		5
4. Streamlit 기본 실행 방법		7
5. 예제 코드		8
6. Streamlit 구성 요소		13



1. 파이썬 GUI

GUI 는 **Graphical User Interface** 의 약자입니다.

사용자가 마우스 클릭, 버튼 선택, 텍스트 입력 등으로 프로그램을 조작할 수 있게 해 줍니다.

예를 들면 다음과 같습니다.

- 로그인 화면
- 계산기 화면
- 데이터 조회 화면
- 이미지 업로드 화면
- AI 예측 결과 출력 화면
- 웹 대시보드

파이썬 GUI 는 크게 두 가지 방향으로 나눌 수 있습니다.

1) 데스크톱 GUI

PC 에 설치해서 실행하는 프로그램 형태

대표 예:

- Tkinter
- PyQt / PySide
- wxPython
- Kivy

2) 웹 GUI

브라우저에서 실행되는 프로그램 형태

대표 예:

- Streamlit
- Gradio
- Dash
- Flask / Django + HTML/CSS/JS



2. Streamlit 이란?

Streamlit 은 파이썬 코드만으로 웹 화면을 쉽게 만들 수 있는 라이브러리입니다.

보통 웹 개발은 다음이 필요합니다.

- HTML
- CSS
- JavaScript
- 백엔드 프레임워크

하지만 Streamlit 은 이런 복잡한 구성을 많이 줄여 줍니다.
파이썬 코드만으로도 다음을 쉽게 만들 수 있습니다.

- 제목
- 버튼
- 입력창
- 파일 업로드
- 표
- 차트
- 이미지 출력
- 모델 예측 결과 표시

즉, 파이썬 초보자도 GUI 프로그램을 빠르게 만들 수 있는 도구입니다.

Streamlit 의 장점

1) 배우기 쉽다

문법이 단순합니다.

예:

```
st.title("나의 첫 앱")  
st.button("클릭")
```

2) 웹 브라우저에서 바로 실행된다

별도 GUI 툴킷 없이 브라우저로 확인할 수 있습니다.

3) 데이터 분석·AI 실습에 적합하다



Pandas, Matplotlib, Scikit-learn 과 잘 어울립니다.

4) 빠르게 프로토타입을 만들 수 있다

직업훈련 수업에서 실습용 결과물을 빠르게 보여주기 좋습니다.

3. 관련 라이브러리

파이썬 GUI 또는 Streamlit 웹 GUI 와 함께 자주 사용하는 라이브러리는 다음과 같습니다.

(1) Streamlit

웹 GUI 를 만드는 핵심 라이브러리

설치:

```
pip install streamlit
```

(2) Pandas

표 형태 데이터 처리

설치:

```
pip install pandas
```

용도:

- CSV 읽기
- 데이터프레임 출력
- 통계 요약

(3) Matplotlib

그래프 출력

설치:

```
pip install matplotlib
```



용도:

- 선 그래프
- 막대 그래프
- 산점도

(4) NumPy

수치 계산

설치:

```
pip install numpy
```

(5) Pillow

이미지 처리

설치:

```
pip install pillow
```

용도:

- 이미지 열기
- 이미지 표시

(6) Scikit-learn

머신러닝 모델 연동

설치:

```
pip install scikit-learn
```

용도:

- 분류/회귀 모델 실습
- 예측 결과를 GUI 에서 표시



4. Streamlit 기본 실행 방법

설치

`pip install streamlit`

실행

예를 들어 `app.py` 파일을 만들었다면:

`streamlit run app.py`

실행하면 브라우저에서 자동으로 열립니다.

Streamlit 기본 구성 요소

자주 쓰는 함수 예시는 다음과 같습니다.

```
import streamlit as st

st.title("제목")
st.header("헤더")
st.subheader("소제목")
st.text("일반 텍스트")
st.write("출력")

입력 요소:

name = st.text_input("이름 입력")
age = st.number_input("나이 입력", min_value=0, max_value=100)
gender = st.selectbox("성별 선택", ["남", "여"])
agree = st.checkbox("동의합니다")
btn = st.button("확인")

출력 요소:

st.success("성공 메시지")
st.warning("경고 메시지")
st.error("오류 메시지")
```



5. 예제 코드

예제 1 : 기본 입력형 GUI

아래 코드는 이름, 나이, 전공을 입력받아 결과를 출력하는 간단한 Streamlit 웹 GUI 입니다.

```
import streamlit as st

st.set_page_config(page_title="파이썬 GUI 예제", page_icon="📄", layout="centered")

st.title("파이썬 Streamlit GUI 예제")
st.write("사용자 입력을 받아 화면에 출력하는 웹 GUI 입니다.")

name = st.text_input("이름을 입력하세요")
age = st.number_input("나이를 입력하세요", min_value=0, max_value=120, step=1)
major = st.selectbox("전공을 선택하세요", ["컴퓨터공학", "정보통신", "전자공학", "AI", "기타"])
memo = st.text_area("자기소개를 입력하세요")

if st.button("확인"):
    if name.strip() == "":
        st.warning("이름을 입력하세요.")
    else:
        st.success("입력이 완료되었습니다.")
        st.write("### 입력 결과")
        st.write(f"이름: {name}")
        st.write(f"나이: {age}")
        st.write(f"전공: {major}")
        st.write(f"자기소개: {memo}")
```

예제 2: 표와 그래프가 포함된 GUI

```
import streamlit as st
import pandas as pd
import matplotlib.pyplot as plt

st.set_page_config(page_title="데이터 시각화 앱", page_icon="📊", layout="wide")
```




```
st.title("Streamlit 데이터 시각화 예제")

data = {
    "과목": ["Python", "SQL", "Spring Boot", "React", "AI"],
    "점수": [85, 78, 92, 88, 95]
}

df = pd.DataFrame(data)

st.subheader("1. 데이터 표")
st.dataframe(df, use_container_width=True)

st.subheader("2. 데이터 요약")
st.write(df.describe(include="all"))

st.subheader("3. 막대그래프")
fig, ax = plt.subplots()
ax.bar(df["과목"], df["점수"])
ax.set_xlabel("과목")
ax.set_ylabel("점수")
ax.set_title("과목별 점수")
st.pyplot(fig)
```

예제 3: 파일 업로드 GUI

```
import streamlit as st
import pandas as pd

st.set_page_config(page_title="CSV 업로드 앱", page_icon="📁")

st.title("CSV 파일 업로드 예제")

uploaded_file = st.file_uploader("CSV 파일을 업로드하세요", type=["csv"])

if uploaded_file is not None:
```



```
df = pd.read_csv(uploaded_file)
st.success("파일 업로드 성공")
st.write("### 데이터 미리보기")
st.dataframe(df, use_container_width=True)

st.write("### 기본 통계")
st.write(df.describe())
else:
    st.info("아직 파일이 업로드되지 않았습니다.")
```

예제 4: Streamlit GUI

기능은 다음과 같습니다.

- 제목 및 설명
- 사용자 정보 입력
- 과목 점수 입력
- 평균 계산
- 표 출력
- 그래프 출력
- 학습 의견 출력

```
import streamlit as st
import pandas as pd
import matplotlib.pyplot as plt

st.set_page_config(
    page_title="AI 직업훈련 학습관리 GUI",
    page_icon="🤖",
    layout="wide"
)

st.title("AI 직업훈련 학습관리 GUI")
st.write("Streamlit 을 활용한 웹 GUI 예제입니다.")

st.sidebar.header("학습자 정보 입력")
```



```
student_name = st.sidebar.text_input("이름")
course = st.sidebar.selectbox("과정 선택", ["Python 기초", "웹 개발", "AI 개발", "데이터 분석"])
attendance = st.sidebar.slider("출석률", 0, 100, 80)

st.header("과목 점수 입력")
col1, col2, col3 = st.columns(3)

with col1:
    python_score = st.number_input("Python 점수", min_value=0, max_value=100, value=85)
with col2:
    sql_score = st.number_input("SQL 점수", min_value=0, max_value=100, value=80)
with col3:
    ai_score = st.number_input("AI 점수", min_value=0, max_value=100, value=90)

if st.button("학습 결과 분석"):
    scores = {
        "과목": ["Python", "SQL", "AI"],
        "점수": [python_score, sql_score, ai_score]
    }

    df = pd.DataFrame(scores)
    avg_score = df["점수"].mean()

    st.subheader("1. 학습자 정보")
    st.write(f"이름: {student_name if student_name else '미입력'}")
    st.write(f"과정: {course}")
    st.write(f"출석률: {attendance}%")

    st.subheader("2. 점수 데이터")
    st.dataframe(df, use_container_width=True)

    st.subheader("3. 평균 점수")
    st.write(f"평균 점수: {avg_score:.2f}")

    st.subheader("4. 점수 그래프")
    fig, ax = plt.subplots()
```



```
ax.bar(df["과목"], df["점수"])
ax.set_xlabel("과목")
ax.set_ylabel("점수")
ax.set_title("과목별 점수")
st.pyplot(fig)

st.subheader("5. 학습 피드백")
if avg_score >= 90:
    st.success("매우 우수한 성과입니다. 심화 프로젝트를 진행해도 좋습니다.")
elif avg_score >= 75:
    st.info("양호한 성과입니다. 실습을 조금 더 강화하면 좋습니다.")
else:
    st.warning("기초 복습과 반복 실습이 필요합니다.")
else:
    st.info("왼쪽 입력과 점수 입력 후 '학습 결과 분석' 버튼을 눌러주세요.")
```

실행 방법

예를 들어 위 코드를 streamlit_app.py 로 저장한 뒤:

```
streamlit run streamlit_app.py
```

실행하면 웹 브라우저에서 GUI 가 열립니다.



6. Streamlit 구성요소(Component)

[Streamlit 공식 문서](#) 참조

1. Streamlit 구성 개요

Streamlit 은 Python 코드만으로 웹 애플리케이션을 구성할 수 있도록 다양한 UI 구성요소(Component)를 제공합니다.

대표 구성요소는 다음과 같습니다.

구분	주요 기능
텍스트 출력	제목, 문장, 코드 출력
입력 컴포넌트	버튼, 입력창, 체크박스
데이터 표시	테이블, 데이터프레임
차트/그래프	선그래프, 바차트
미디어	이미지, 오디오, 비디오
레이아웃	컬럼, 사이드바, 탭
상태관리	session_state
파일 처리	파일 업로드/다운로드
채팅 UI	AI 챗봇 인터페이스
캐시	성능 최적화

2. 텍스트 출력 구성요소

2-1. 제목

```
import streamlit as st

st.title("AI 데이터 분석 시스템")
```

출력:

AI 데이터 분석 시스템

2-2. 헤더

```
st.header("회원 관리")
```



2-3. 서브헤더

```
st.subheader("로그인 정보")
```

2-4. 일반 텍스트

```
st.text("텍스트 출력")
```

2-5. Markdown

```
st.markdown("## Markdown 지원")
```

Markdown 문법 사용 가능

- 제목
- 리스트
- 코드블록
- 링크

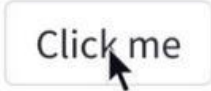
2-6. 코드 출력

```
code = """  
print("hello")  
"""
```

```
st.code(code, language="python")
```



3. 입력(Input) 구성요소

		
Button Display a button widget.	Download button Display a download button widget.	Checkbox Display a checkbox widget.
<pre>clicked = st.button("Click me")</pre>	<pre>st.download_button("Download fi</pre>	<pre>selected = st.checkbox("I agree</pre>
		
Radio Display a radio button widget.	Selectbox Display a select widget.	Multiselect Display a multiselect widget. The multiselect widget starts as empty.
<pre>choice = st.radio("Pick one", [</pre>	<pre>choice = st.selectbox("Pick one</pre>	<pre>choices = st.multiselect("Buy",</pre>

3-1. 버튼

```
if st.button("저장"):
    st.write("저장 완료")
```

3-2. 텍스트 입력

```
name = st.text_input("이름 입력")
```



3-3. 숫자 입력

```
age = st.number_input("나이")
```

3-4. 비밀번호 입력

```
password = st.text_input(  
    "비밀번호",  
    type="password"  
)
```

3-5. 체크박스

```
agree = st.checkbox("동의합니다")
```

3-6. 라디오 버튼

```
gender = st.radio(  
    "성별 선택",  
    ["남성", "여성"]  
)
```

3-7. 셀렉트박스

```
job = st.selectbox(  
    "직업 선택",  
    ["학생", "개발자", "강사"]  
)
```

3-8. 멀티 셀렉트

```
subjects = st.multiselect(  
    "과목 선택",  
    ["Python", "AI", "DB"]  
)
```




3-9. 슬라이더

```
score = st.slider(  
    "점수",  
    0,  
    100  
)
```

3-10. 날짜 입력

```
date = st.date_input("날짜 선택")
```

4. 데이터 표시 구성요소

4-1. 데이터프레임

```
import pandas as pd
```

```
df = pd.DataFrame({  
    "이름": ["홍길동", "김철수"],  
    "점수": [90, 80]  
})
```

```
st.dataframe(df)  
스크롤 가능 테이블
```

4-2. 정적 테이블

```
st.table(df)
```

4-3. 메트릭

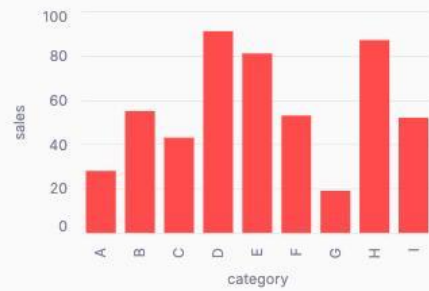
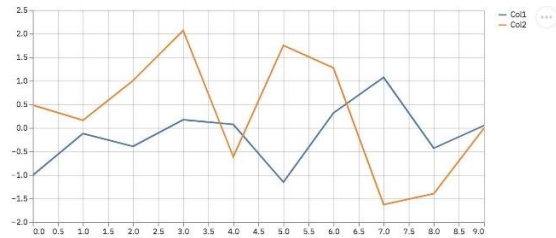
```
st.metric(  
    label="총 사용자",  
    value=120  
)
```



5. 차트(Graph) 구성요소

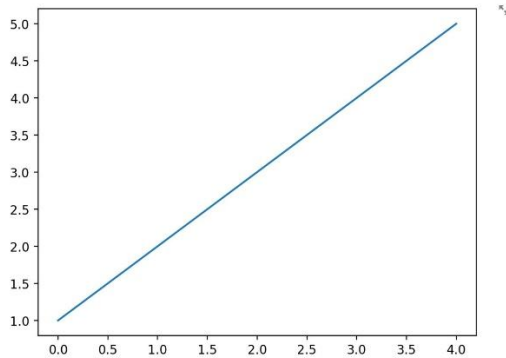
Welcome to Streamlit!

Line Chart in Streamlit



```
st.bar_chart(df, x="category", y="sales")
```

A simple figure with st.pyplot!



5-1. 선 그래프

```
st.line_chart(df)
```

5-2. 막대 그래프

```
st.bar_chart(df)
```

5-3. 영역 그래프

```
st.area_chart(df)
```



5-4. matplotlib 그래프

```
import matplotlib.pyplot as plt
```

```
fig, ax = plt.subplots()
ax.plot([1,2,3], [4,5,6])
st.pyplot(fig)
```

6. 미디어 구성요소

6-1. 이미지

```
st.image("test.png")
```

6-2. 오디오

```
st.audio("music.mp3")
```

6-3. 비디오

```
st.video("movie.mp4")
```

7. 파일 처리 구성요소

7-1. 파일 업로드

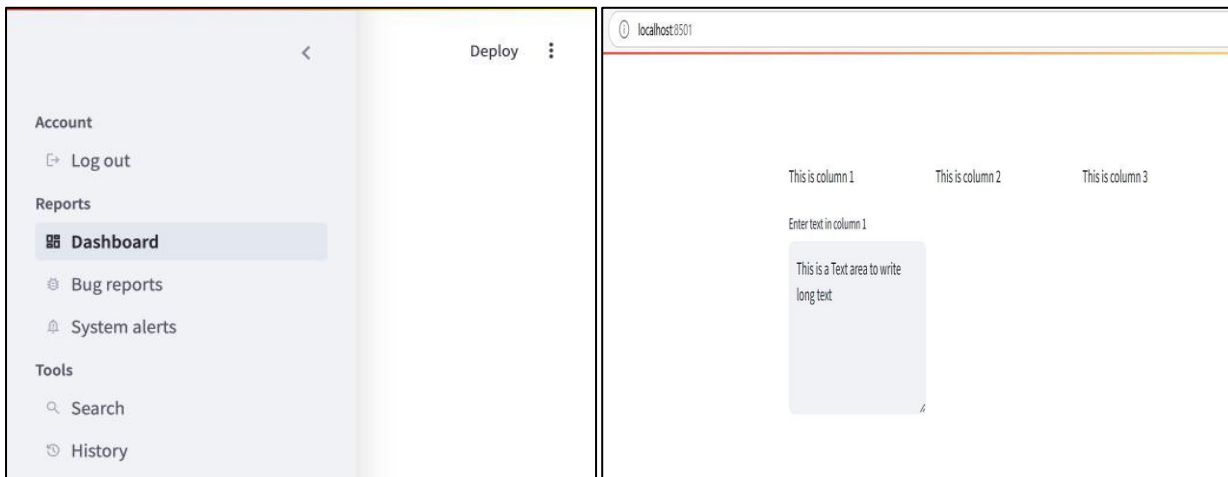
```
uploaded_file = st.file_uploader(
    "파일 업로드",
    type=["csv"]
)
```

7-2. 다운로드 버튼

```
st.download_button(
    label="다운로드",
    data="hello",
    file_name="test.txt"
)
```



8. 레이아웃(Layout) 구성요소



st.expander

v1.17.0

Insert a multi-element container that can be expanded/collapsed.

Inserts a container into your app that can be used to hold multiple elements and can be expanded or collapsed by the user. When collapsed, all that is visible is the provided label.

To add elements to the returned container, you can use "with" notation (preferred) or just call methods directly on the returned object. See examples below.

Warning

Currently, you may not put expanders inside another expander.

Function signature

```
st.expander(label, expanded=False)
```

Parameters

label (<i>str</i>)	A string to use as the header for the expander. The label can optionally contain Markdown and supports the following elements: Bold, Italics, Strikethroughs, Inline Code, Emojis and Links.
expanded (<i>bool</i>)	If True, initializes the expander in "expanded" state. Defaults to False (collapsed).

8-1. 사이드바

```
st.sidebar.title("메뉴")
```

왼쪽 메뉴 영역 생성



8-2. 컬럼

```
col1, col2 = st.columns(2)
```

```
with col1:
```

```
    st.write("왼쪽")
```

```
with col2:
```

```
    st.write("오른쪽")
```

8-3. 탭

```
tab1, tab2 = st.tabs(["회원", "게시판"])
```

8-4. 확장 패널

```
with st.expander("상세정보"):
```

```
    st.write("내용")
```

9. 상태관리(Session State)

Streamlit 은 새로고침 시 변수값이 초기화됩니다.

이를 유지하기 위해 사용:

```
if "count" not in st.session_state:
```

```
    st.session_state.count = 0
```

```
st.session_state.count += 1
```

10. 폼(Form)

```
with st.form("login_form"):
```

```
    email = st.text_input("이메일")
```

```
    password = st.text_input(
        "비밀번호",
```

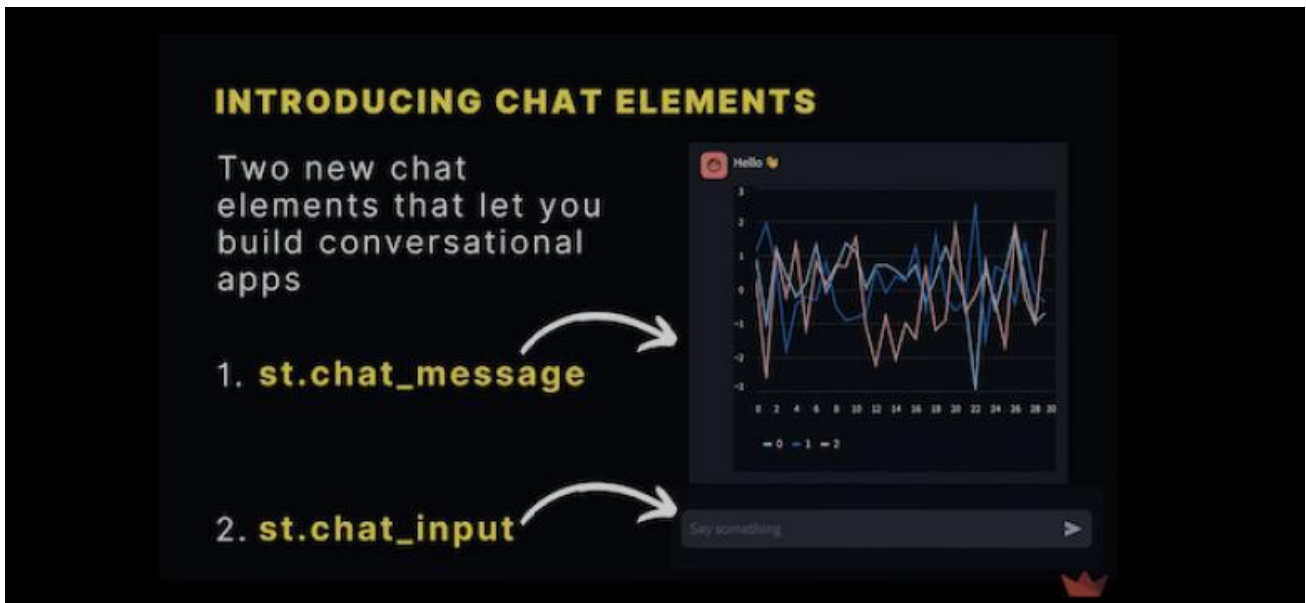


```
type="password"  
)
```

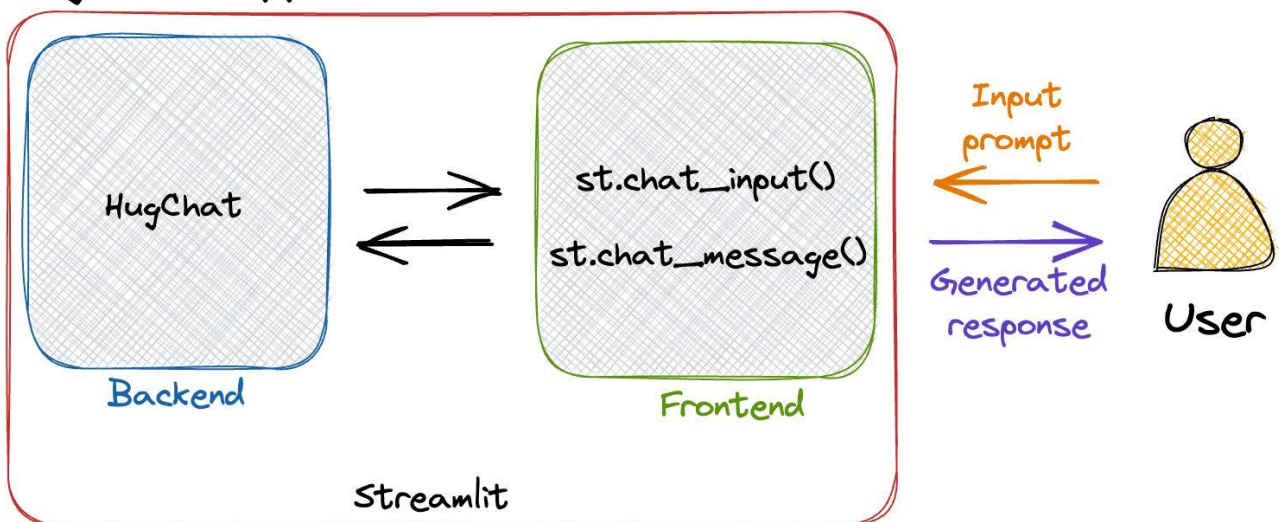
```
submit = st.form_submit_button("로그인")
```

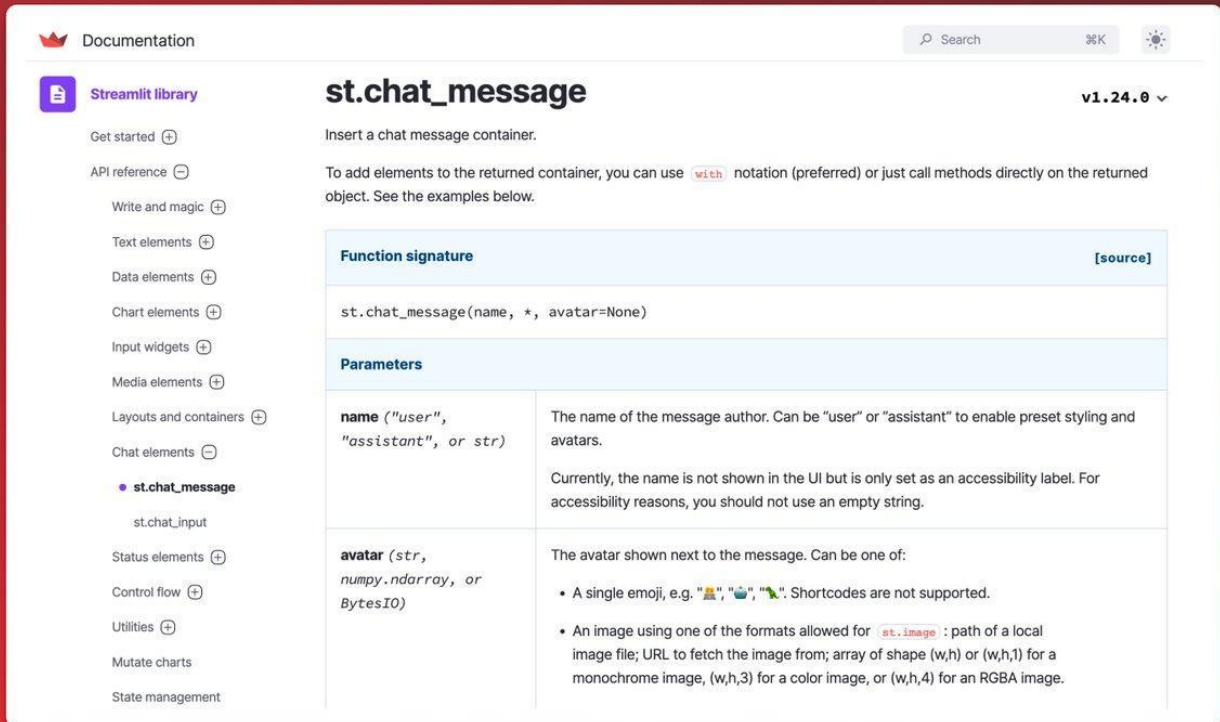
폼 단위 입력 처리 가능

11. AI 챗봇 구성요소



HugChat app





11-1. 채팅 입력

```
prompt = st.chat_input("질문 입력")
```

11-2. 채팅 메시지

```
with st.chat_message("user"):
    st.write("안녕하세요")
```

12. 캐시(Cache)

AI/데이터 처리 속도 개선에 매우 중요

12-1. 데이터 캐시

```
@st.cache_data
def load_data():
    return pd.read_csv("data.csv")
```



12-2. 리소스 캐시

```
@st.cache_resource
```

```
def load_model():
```

```
    return model
```

AI 모델 로딩 최적화

13. Streamlit AI 서비스 구성 예시

AI 챗봇

사용자 입력

↓

Streamlit UI

↓

OpenAI / FastAPI 호출

↓

LLM 응답

↓

결과 출력

데이터 분석 서비스

CSV 업로드

↓

Pandas 분석

↓

통계 계산

↓

그래프 출력



14. Streamlit 주요 특징

특징	설명
Python 만 사용	HTML/CSS 최소화
빠른 개발	AI 데모 제작 최적
데이터 친화적	Pandas/Numpy 연동
AI 친화적	OpenAI/LangChain 연동 쉬움
실시간 반응	입력 즉시 UI 갱신
쉬운 배포	Cloud 배포 가능

15. Streamlit + AI 분야 활용 예

분야	구현 가능 서비스
생성형 AI	챗봇
머신러닝	예측 시스템
딥러닝	이미지 분석
RAG	문서 검색
데이터 분석	BI 대시보드
교육	실습 플랫폼
업무자동화	사내 AI 툴

16. 실무에서 자주 사용하는 라이브러리 조합

라이브러리	용도
Pandas	데이터 처리
NumPy	수치 계산
Matplotlib	그래프
Plotly	인터랙티브 차트
Scikit-learn	머신러닝
TensorFlow	딥러닝
LangChain	LLM 연동
FastAPI	AI 서버
OpenAI Platform	GPT API